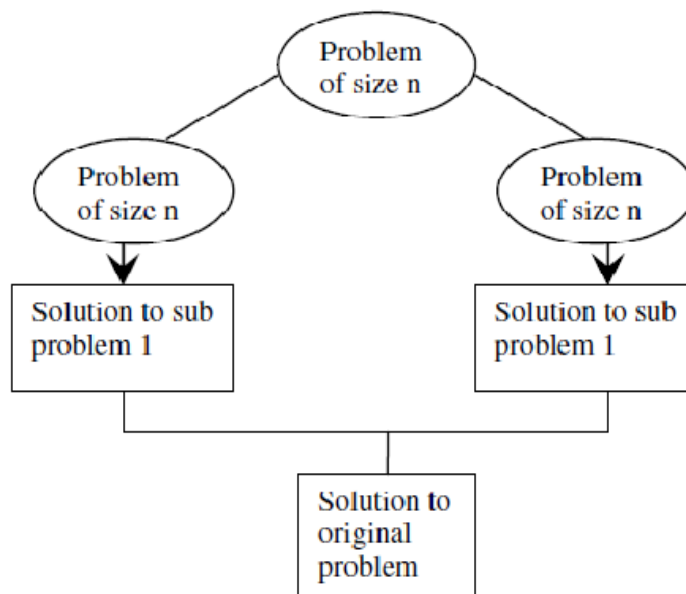


ASSIGNMENT II :- SOLVED

- **Divide and Conquer** : It is a top-down technique for designing algorithms that consists of dividing the problem into smaller sub-problems hoping that the solutions of the sub-problems are easier to find & then combine the partial solutions onto the solutions of the original problems.
- **General Method** : The general method in solving a given problem is shown below:
 - An instance of a given problem is divided into a number of smaller instances of same type & equal size.
 - All the smaller instances of the problems are solved recursively.
 - The solutions of all the smaller instances are combined together to get a solution to the original problem.
- Given a function to compute on n inputs the divide-and-conquer strategy suggests splitting the inputs into k distinct subsets, $1 < k \leq n$, yielding k sub-problems
- These sub-problems must be solved, and then a method must be found to combine sub-solutions into a solution of the whole
- If the sub-problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied
- Often the sub-problems resulting from a divide-and-conquer design are of the same type as the original problem
- Similarly, smaller and smaller sub-problems of the same kind are generated until no more splitting of sub-problem is possible

Divide and Conquer Technique:



- The general algorithm for divide and conquer (DAC) method is as follows:

Algorithm DAC(P)

//Purpose : Solve the problem of a given instance(P) by dividing into various

//smaller instances such as p1,p2,p3pn.

//Input: The instances of a problem are (P)

//output : The solution S to the input instances

if small (P)

return G(P)

else

Divide P into p1, p2,.....,pk

S ← DAC(p1) + DAC(p2)+.....+DAC(pk)

return S

end if

Merge Sort : Introduction

- Given a sequence of elements $a[1].....a[n]$.
- The general idea here is to split them into two sets $a[1].....a[n/2]$ and $a[n/2+1].....a[n]$.
- Each set is individually sorted and the resulting sorted sequences are merged to produce a single sorted sequence of n elements.

Working :

- Divide the array into equal parts.
- Sort the left part of the array recursively.
- Sort the right part of the array recursively.

Merge the left part & right part by comparing the elements & placing the lowest elements first in the resultant array.

Algorithm merge_sort (L) :

```
{
// Input List or Array L
if len(L) > 1 :
    mid = len(L) / 2
    LH = L[ 0 : mid]
    RH = L[mid: high]

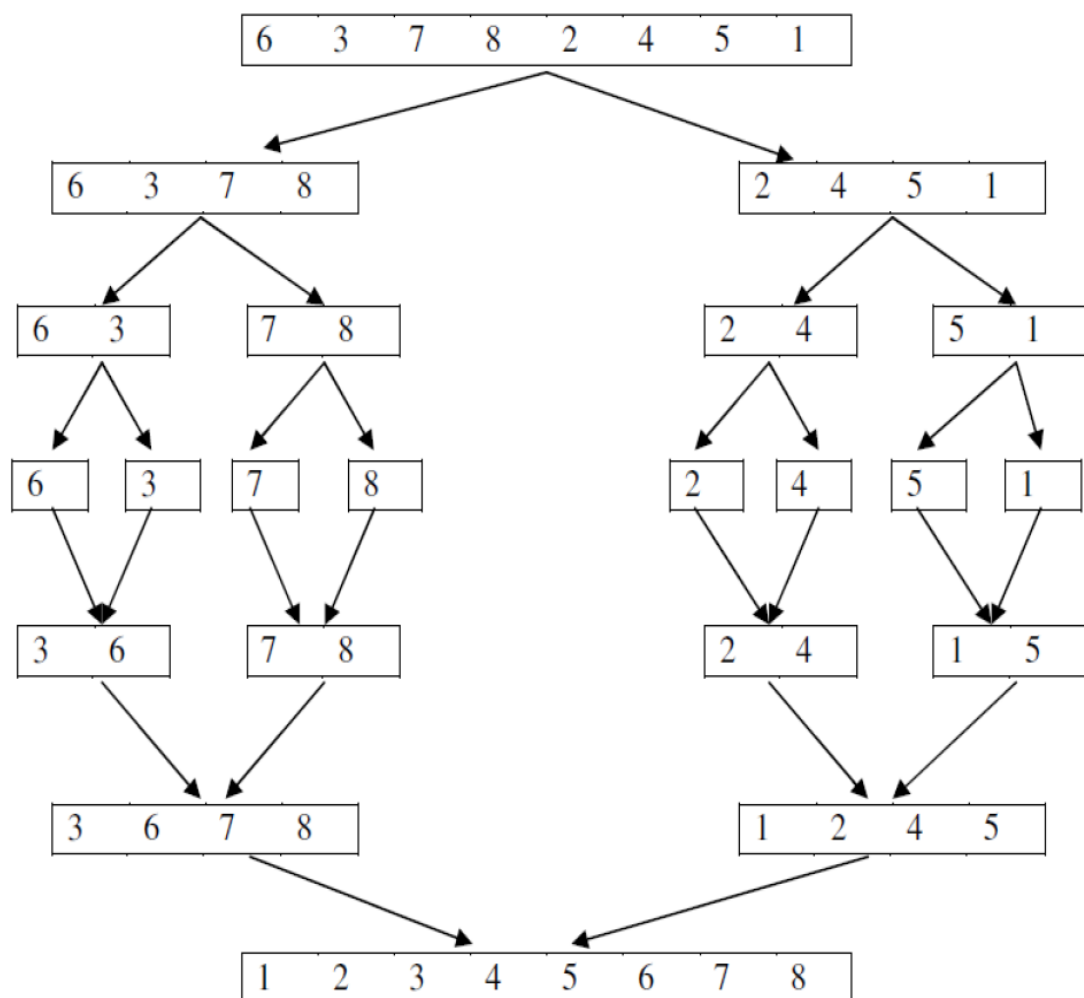
    merge_sort ( LH )
    merge_sort ( RH )
    merge ( LH , RH )
}
```

Algorithm merge (LH , RH) :

```
{  
// Input : LH – Left Hand List RH – Right Hand List  
i = j = k = 0  
  
while i < len(LH) and j < len(RH) :  
    if LH[i] < RH[j]:  
        L[k]=LH[i]  
        i+=1  
    else:  
        L[k]=RH[j]  
        j+=1  
        k+=1  
  
while i < len(LH):  
    L[k]=LH[i]  
    i+=1  
    k+=1  
while j < len(RH):  
    L[k]=RH[j]  
    j+=1  
    k+=1  
}
```

Example:

- Apply merge sort for the following list of elements : 6, 3, 7, 8, 2, 4, 5, 1



Quick Sort - (Partition Exchange Sort)

Introduction :

- Pick an element called pivot from the list.
- Reorder the list so that all elements which are less than the pivot come before the pivot and all elements greater than pivot come after it.
- After this partitioning, the pivot is in its final position. This is called the partition operation.
- Recursively sort the sub-list of lesser elements and sub-list of greater elements

Follow the 4 steps for quick sort:

- If $A[\text{pivot}] > A[i]$, increment i .
- If $A[\text{pivot}] < A[j]$, decrement j .
- If we can not increment i or decrement j , swap $A[i]$ and $A[j]$
When $j \leq i$, swap $A[j]$ and $A[\text{pivot}]$

ALGORITHM of QUICK-SORT

Algorithm quick_sort(L, low, high):

if low < high :

 pi = partition(L, low, high)

 quick_sort(L, low, pi - 1)

 quick_sort(L, pi + 1 , high)

Algorithm partition(L, low, high):

 pivot = L[low]

 i = low + 1

 j = high

 while True:

 while i <= j and L[i] < pivot :

 i+=1

 while i <= j and L[j] > pivot :

 j-=1

 if i < j :

 L[i], L[j] = L[j] , L[i]

 else:

 break

 L[low] , L[j] = L[j], L[low]

 return j

Example

Indices : 0, 1, 2, 3, 4, 5, 6, 7

List of values : 5, 3, 1, 9, 8, 2, 4, 7

0	1	2	3	4	5	6	7
5	3	1	9	8	2	4	7
5	3	1	9	8	2	4	7
5	3	1	4	8	2	9	7
5	3	1	4	8	2	9	7
5	3	1	4	2	8	9	7
5	3	1	4	2	8	9	7
2	3	1	4	5	8	9	7
2	3	1	4				
2	3	1	4				
2	1	3	4				
2	1	3	4				
1	2	3	4				
1		3	4				
		3	4				
					8	9	7
					8	7	9
					8	7	9
					7	8	9
					7		9

OUTPUT : Sorted listion Ascending order i.e 1, 2, 3, 4, 5, 7, 8, 9

Insertion Sort :

- Consider an array of n elements. We assume that a[i] is the item to be inserted and assign it to item, for 0 < i < n.
- Compare the item a[i] from position (i-1) down to 0 and insert into appropriate place i.e the item should be compared with a[j], i-1≥j ≥0 where initial value of j is i-1.
- As long as the item is less than a[j], copy a[j] to a[j+1] and decrement j by 1.
- As we decrement j by 1, we should see that j should be greater than or equal to zero.
- When item is greater than a[j] or j < 0, insert item into a[j+1].
- Repeat the above procedure for each item of the array.

Algorithm InsertionSort(a,n)

```
{
  for i ← 1 to n-1 do
    item ← a[i]
    j ← i-1
    while(j ≥ 0 and item < a[j])
      a[j+1] ← a[j]
      j ← j-1
    end while
    a[j+1] ← item
  end for
}
```

Depth-First Search :

- Depth-first search starts visiting vertices of a graph at an arbitrary vertex by marking it as having been visited.
- On each iteration, the algorithm proceeds to an unvisited vertex that is adjacent to the one it is currently in.
- This process continues until a dead end—a vertex with no adjacent unvisited vertices—is encountered.
- At a dead end, the algorithm backs up one edge to the vertex it came from and tries to continue visiting unvisited vertices from there.
- The algorithm eventually halts after backing up to the starting vertex, with the latter being a dead end.
- By then, all the vertices in the same connected component as the starting vertex have been visited.
- If unvisited vertices still remain, the depth-first search must be restarted at any one of them.
- It is convenient to use a stack to trace the operation of depth-first search.
- We push a vertex onto the stack when the vertex is reached for the first time (i.e., the visit of the vertex starts), and we pop a vertex off the stack when it becomes a dead end (i.e., the visit of the vertex ends).

ALGORITHM *DFS*(G)

mark each vertex in V with 0 as a mark of being "unvisited"

$Count \leftarrow 0$

for each vertex v in V do

 if v is marked with 0

 dfs (v)

```

dfs(v)
//visits recursively all the unvisited vertices
//connected to vertex  $v$  by a path and
//numbers them in the order they are
//encountered via global variable  $count$ 
 $count \leftarrow count + 1$ ;
mark  $v$  with  $count$ 
for each vertex  $w$  in  $V$  adjacent to  $v$  do
    if  $w$  is marked with 0
        dfs(w)

```

DFS Implementation using PYTHON programming

```

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            print(node, end=" , ")
            visited.add(node)
            # Push neighbors in reverse order to maintain correct DFS order
            stack.extend(reversed(graph[node]))

# Example graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

print("\nDFS (Iterative) Traversal Starting from Node A :")
dfs_iterative(graph, 'A')

```

-----OUTPUT-----

DFS (Iterative):

A , B, D, E, F, C

NOTE : DRAW THE GRAPH

Breadth First Search (BFS) is a fundamental **graph traversal algorithm**. It begins with a node, then first traverses all its adjacent nodes. Once all adjacent are visited, then their adjacent are traversed.

BFS from a Given Source

The algorithm starts from a given source and explores all reachable vertices from the given source. It is similar to the [Breadth-First Traversal of a tree](#). Like tree, we begin with the given source (in tree, we begin with root) and traverse vertices level by level using a queue data structure. The only catch here is that, unlike trees, graphs may contain cycles, so we may come to the same node again.

```
def BFS (graph, start) :
```

```
visited = []
```

Q = [start]

while Q :

```
node = Q.pop(0)
```

if node not in visited :

```
print(node, end=' ')
```

```
visited.append(node)
```

```
Q.extend( graph[node] )
```

```
graph = { 'A': ['B','C'], 'B': ['D','E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': [] }
```

```
print('BFS Traversal starting from node A :')
```

```
BFS(graph, 'A')
```

-----OUTPUT-----

BFS Traversal starting from node A :

A B C D E F

NOTE : DRAW THE GRAPH

Binary Tree Traversals

- The most important divide-and-conquer algorithms for binary trees are the three classic traversals: preorder, inorder, and postorder.
- All three traversals visit nodes of a binary tree recursively, i.e., by visiting the tree's root and its left and right subtrees.

They differ only by the timing of the root's visit:

- In the **preorder traversal**, the root is visited before the left and right subtrees are visited (in that order).
- In the, the root is visited after visiting its left subtree but before visiting the **inorder traversal** right subtree.

- In the **postorder traversal**, the root is visited after visiting the left and right subtrees (in that order).

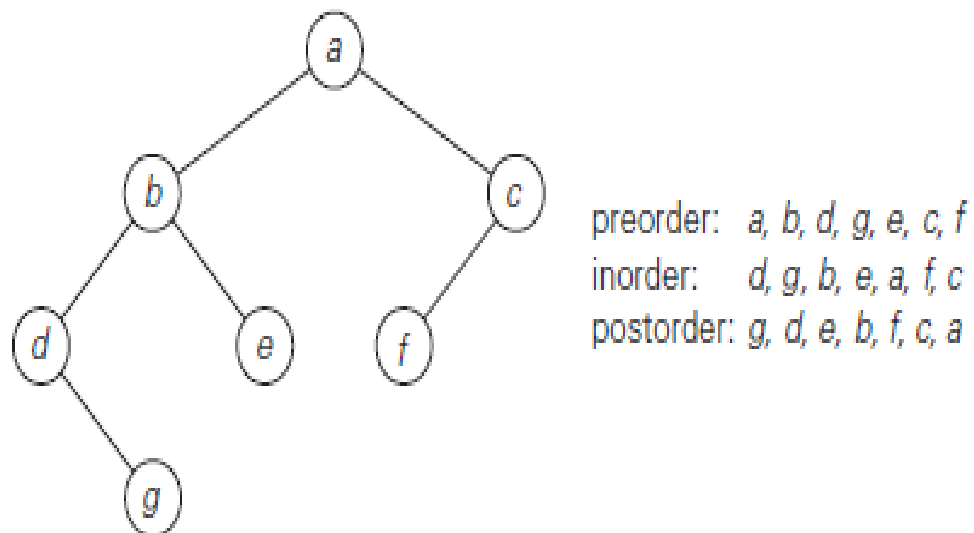


FIGURE 5.6 Binary tree and its traversals.

Binary Search is an searching algorithm that operates on a sorted or monotonic search space, repeatedly dividing it into halves to find a target value or optimal answer in logarithmic time $O(\log N)$.

Conditions to apply Binary Search Algorithm in a Data Structure

To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure should take constant time.

Binary Search Algorithm

Below is the step-by-step algorithm for Binary Search:

- Divide the search space into two halves by finding the middle index "mid".
- Compare the middle element of the search space with the key.
- If the key is found at middle element, the process is terminated.
- If the key is not found at middle element, choose which half will be used as the next search space.
 - > If the key is smaller than the middle element, then the left side is used for next search.
 - > If the key is larger than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

Algorithm Bin_Search(L,low,high,Target):

if low > high :

return -1

else:

mid = (low+high) //2

if L[mid] == Target:

return mid

```
elif Target < L[mid]:  
    return Bin_Search(L,0,mid-1,Target)  
else:  
    return Bin_Search(L,mid+1,high,Target)
```

```
L = [1,3,5,7,9,11,13,15]
```

```
Target = 17
```

```
result = Bin_Search(L,0,len(L)-1, Target)
```

```
if result == -1 :
```

```
    print 'The SERACH is UNSUCCESSFUL '
```

```
else:
```

```
    print 'The SERACH is SUCCESSFUL and Target is found at',result + 1
```